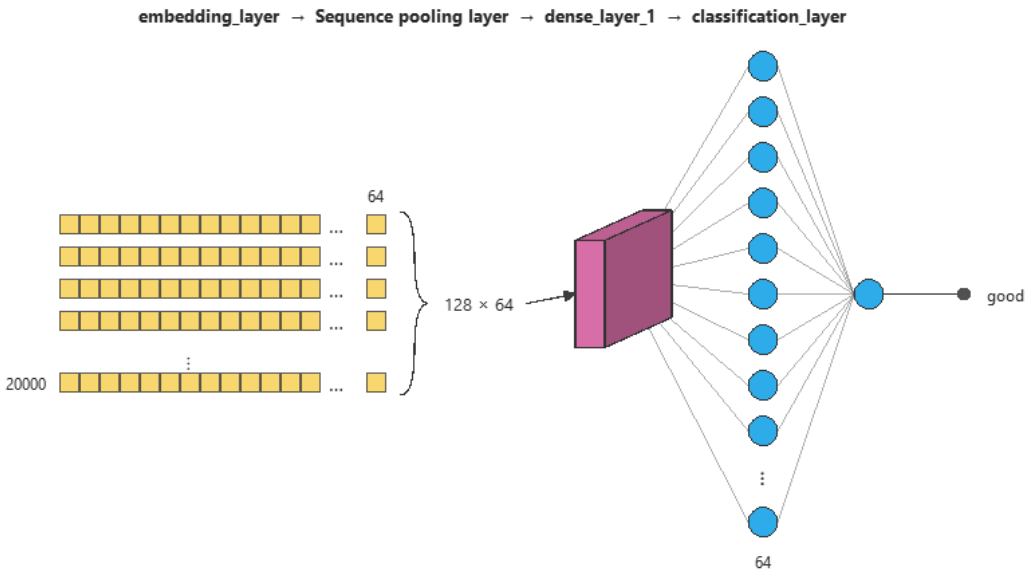
impacto

Qué opinan los clientes de un negocio

Reseñas de Google clasificadas como buenas o malas con un modelo de texto entrenado en Neural Designer, ejecutado con OpenNN y desplegado con Docker



Miguel Barranquero Diez

GitHub github.com/miguelbdiez
Código github.com/miguelbdiez/impacto
LinkedIn linkedin.com/in/miguel-barranquero-diez

Octubre de 2026

Resumen

impacto recoge las reseñas públicas de Google de un negocio, clasifica cada una como buena o mala con un modelo de texto en español y muestra los resultados en una aplicación web y en un informe de Power BI: cómo evolucionan las opiniones, si el modelo coincide con las estrellas y de qué hablan las reseñas malas y las buenas.

El modelo se entrenó en **Neural Designer** con 80.000 reseñas de Amazon en español y se ejecuta con un pequeño programa en C++ basado en **OpenNN**, sin el motor completo de la aplicación. El caso de estudio son 7.490 reseñas de El Corte Inglés Princesa (Madrid) publicadas entre octubre de 2021 y octubre de 2026. Todo el sistema, con la base de datos PostgreSQL, arranca con un solo comando de Docker. El proyecto no ha costado nada: solo se han usado herramientas libres y planes gratuitos.

88,5 %	87,1 %	7.490	0 €
exactitud en reseñas que el modelo no había visto	coincidencia con las estrellas en reseñas reales	reseñas de Google analizadas	de coste, solo planes gratuitos

Contenido

1. **Introducción y objetivos** 2
2. **Arquitectura y tecnologías** 3
3. **Datos de entrenamiento** 4
4. **Entrenamiento con Neural Designer** 4
5. **Inferencia con OpenNN** 7
6. **Recogida de reseñas** 8
7. **Base de datos y pipeline** 8
8. **Resultados en El Corte Inglés Princesa** 9
9. **Aplicación web** 10
10. **Informe de Power BI** 13
11. **Despliegue con Docker** 13
12. **Conclusiones y trabajo futuro** 14

Licencia, datos y reproducibilidad 15

1 Introducción y objetivos

1.1 Motivación

La nota media de Google resume en un número miles de opiniones, pero no dice qué está cambiando ni por qué. Un negocio con 4,2 estrellas puede tener cada año más clientes descontentos sin que la media apenas se mueva. Leer las reseñas una a una no escala.

Este proyecto clasifica el *texto* de cada reseña con un modelo de aprendizaje automático. Las estrellas ya existen, pero el modelo de texto sirve para algo más: en el futuro podrá clasificar opiniones que no llevan estrellas, como mensajes de foros o de redes sociales. Mientras tanto, las estrellas de Google sirven para comprobar gratis si el modelo acierta en un dominio real.

Es un proyecto de portfolio y recorre el ciclo completo: datos de entrenamiento, entrenamiento y evaluación del modelo, despliegue de la inferencia, recogida de datos reales, almacenamiento, análisis, visualización y despliegue. El modelo se entrenó en Neural Designer, la aplicación de machine learning de Artelnics en cuyo desarrollo trabaja el autor, de modo que el proyecto sirve también como caso práctico de la herramienta.

1.2 Objetivos

1. Entrenar en Neural Designer un clasificador de texto en español (*buena* / *mala*) y compararlo con un modelo de referencia clásico.
2. Ejecutar el modelo fuera de Neural Designer con un programa ligero basado en OpenNN, con los mismos resultados.
3. Recoger las reseñas de un negocio real sin coste y sin datos personales de sus autores.
4. Analizar los resultados: evolución en el tiempo, coincidencia con las estrellas y temas de las reseñas.
5. Mostrarlos en una aplicación web (Streamlit) y en un informe de Power BI.
6. Desplegarlo todo con Docker.
7. Documentar cada decisión, con las alternativas y los motivos, en un registro de decisiones.

1.3 Caso de estudio y restricciones

El caso de estudio es **El Corte Inglés Princesa (Edificio Moda)**, en la calle de la Princesa de Madrid: 4,2 estrellas y 13.706 reseñas en Google. Una marca conocida a nivel nacional hace el resultado más cercano, y su número de reseñas cabe en el plan gratuito del servicio de recogida (El Corte Inglés de Castellana, con más de 42.000, se descartó por eso). La aplicación permite analizar cualquier otro negocio.

Este es un análisis educativo de reseñas públicas, sin relación con la empresa. Se muestran cifras agregadas y fragmentos cortos, nunca el conjunto de los textos.

Tres restricciones han guiado el diseño:

- **Coste cero.** Solo herramientas libres y planes gratuitos, sin ningún medio de pago registrado.
- **Privacidad.** No se guardan nombres, fotos ni perfiles de los autores de las reseñas, y nunca se evitan CAPTCHAs ni bloqueos.
- **Licencias de datos.** Los datos de entrenamiento no se pueden redistribuir: el repositorio incluye los scripts que los reconstruyen, no los datos.

2 Arquitectura y tecnologías

El sistema tiene dos partes (figura 1). El **entrenamiento** se hace una vez: reseñas de Amazon, normalización del texto y entrenamiento en Neural Designer, de donde se exporta la red. La **producción** se repite con cada negocio: descarga de reseñas de Google Maps, base de datos, clasificación con el predictor de OpenNN y visualización.

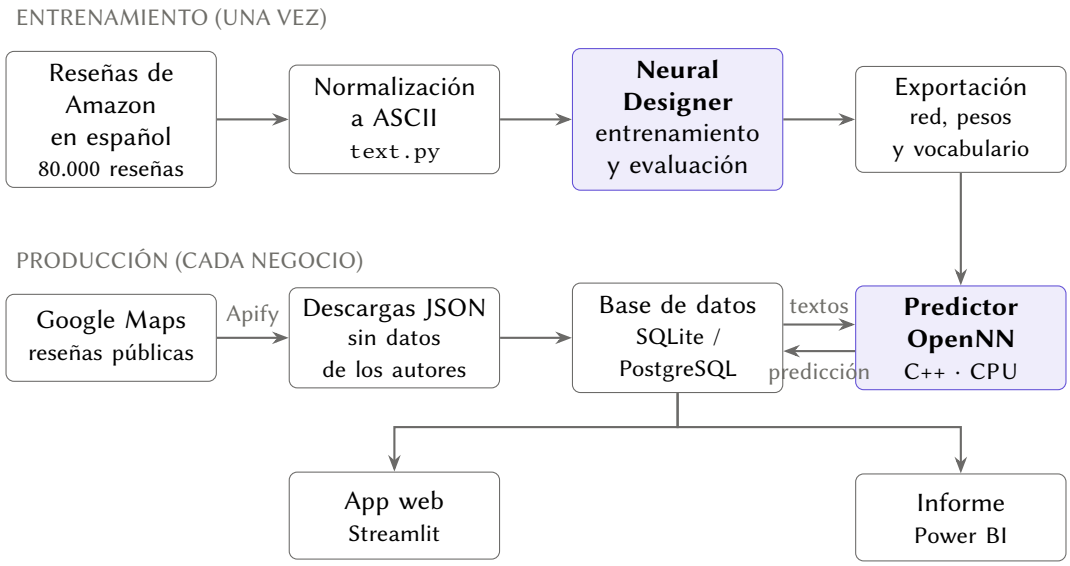


Figura 1. Arquitectura de impacto. En violeta, las dos piezas del modelo: Neural Designer para entrenar y OpenNN para predecir.

Tabla 1. Tecnologías utilizadas.

Etapa	Tecnología	Uso
Datos	Python, pandas, Hugging Face	Descarga y preparación de las reseñas de entrenamiento
Entrenamiento	Neural Designer	Red neuronal, entrenamiento, informes y matriz de confusión
Referencia	scikit-learn	Modelo clásico TF-IDF con regresión logística, para comparar
Inferencia	OpenNN (C++20), CMake	Predictor ligero que ejecuta la red exportada
Recogida	Apify (plan gratuito), Playwright	Descarga de reseñas de Google Maps y búsqueda de negocios
Almacenamiento	SQLAlchemy, SQLite, PostgreSQL 17	Reseñas, descargas y predicciones
Visualización	Streamlit, Altair, Power BI Desktop	Aplicación web, gráficos e informe con DAX y DirectQuery
Despliegue	Docker, Docker Compose	Aplicación y base de datos en contenedores
Entorno	uv, Git, GitHub	Dependencias reproducibles y control de versiones

Registro de decisiones. Todas las decisiones del proyecto están documentadas, con su contexto, las alternativas consideradas y los motivos, en el archivo `docs/decision-log.md` del repositorio. Cuando una decisión cambió, se añadió una actualización con fecha en lugar de reescribirla. Esta memoria resume ese registro.

3 Datos de entrenamiento

3.1 El conjunto de datos

Un solo negocio no tiene reseñas suficientes para entrenar un modelo, y casi todas son de 4 y 5 estrellas. Por eso el modelo se entrena con un conjunto público: la parte en español del *Multilingual Amazon Reviews Corpus* [1], con 200.000 reseñas de productos repartidas por igual entre 1 y 5 estrellas. Su licencia permite el uso no comercial y prohíbe redistribuir los datos, así que el repositorio documenta la fuente, cada paso de procesado y las sumas de comprobación, e incluye los scripts que reconstruyen los archivos.

3.2 De estrellas a etiquetas: dos clases

Las estrellas dan la etiqueta: 1–2 estrellas, *mala*; 4–5, *buena*. Las de 3 estrellas se descartan.

Antes se probó una tercera clase, *neutra*, para las de 3 estrellas. Las redes llegaron al 62,8 % de exactitud y el modelo de referencia al 65,8 %–68,0 %. Casi todos los errores estaban en la clase neutra: solo se reconocía entre el 49 % y el 56 % de las reseñas neutras, mientras que confundir una buena con una mala ocurría en el 4 %–9 % de los casos. Una reseña de 3 estrellas mezcla elogios y quejas, así que las estrellas dan una etiqueta fiable para los extremos y ruidosa para el centro. El modelo final usa dos clases, con 40.000 reseñas de cada una.

Solo el texto de la reseña es entrada del modelo. El título no se usa, porque las reseñas de Google y los mensajes de redes sociales no lo tienen, y las estrellas tampoco, porque de ellas sale la etiqueta.

3.3 Un problema con los acentos

El vocabulario del primer modelo tenía entre sus palabras más frecuentes fragmentos como `m`, `s`, `est` o `peque`. La causa estaba en el tokenizador de palabras de OpenNN: clasifica el texto byte a byte, y en UTF-8 las letras con tilde, la ñ y los signos ¿ ¡ ocupan dos bytes que no cuentan como letras. La palabra se partía en dos (tabla 2). Afectaba al 7,5 % de las palabras del conjunto de datos.

Tabla 2. Cómo partía las palabras el tokenizador de OpenNN antes de normalizar el texto.

Texto	Tokens	Texto	Tokens
más	`m`, `s`	también	`tambi`, `n`
pequeño	`peque`, `o`	mañana	`ma`, `ana`

La solución fue normalizar el texto a ASCII (*más* pasa a *mas*) en una única función, `normalize_text`, que usan tanto la preparación de los datos de entrenamiento como la inferencia. Así el modelo ve exactamente los mismos tokens al entrenar y al predecir. Se pierden distinciones como *año*/*ano*, que no importan para el sentimiento. El arreglo de fondo, un tokenizador que entienda UTF-8, se comunicó al equipo de OpenNN.

4 Entrenamiento con Neural Designer

4.1 Configuración

El archivo de entrenamiento se cargó en un proyecto de clasificación de texto de Neural Designer. Cada columna recibe un papel: el texto es la entrada, la etiqueta es el objetivo y el resto de columnas (identificadores, categoría del producto, estrellas, título) no se usan. Los datos se dividen en 60 % para entrenamiento, 20 % para validación (Neural Designer lo llama *selección*) y 20 % para prueba: 48.000, 16.000 y 16.000 reseñas.

- **Vocabulario:** palabras que aparecen al menos dos veces, hasta 20.000. Las que aparecen una sola vez son sobre todo erratas.
- **Longitud de secuencia:** 128 tokens. La reseña mediana tiene 28 y el 98,6 % cabe en 128.

4.2 Arquitectura de la red

La red seleccionada (figura 2) es sencilla:

1. **Embedding** de 20.000 palabras en 64 dimensiones, con codificación posicional.
2. **Average pooling:** la media de los vectores de la reseña, ignorando el relleno.
3. **Capa densa** de 64 neuronas con activación ReLU.
4. **Salida sigmoide:** la probabilidad de que la reseña sea buena.

Tiene 1.284.225 parámetros, casi todos en la tabla de embedding.

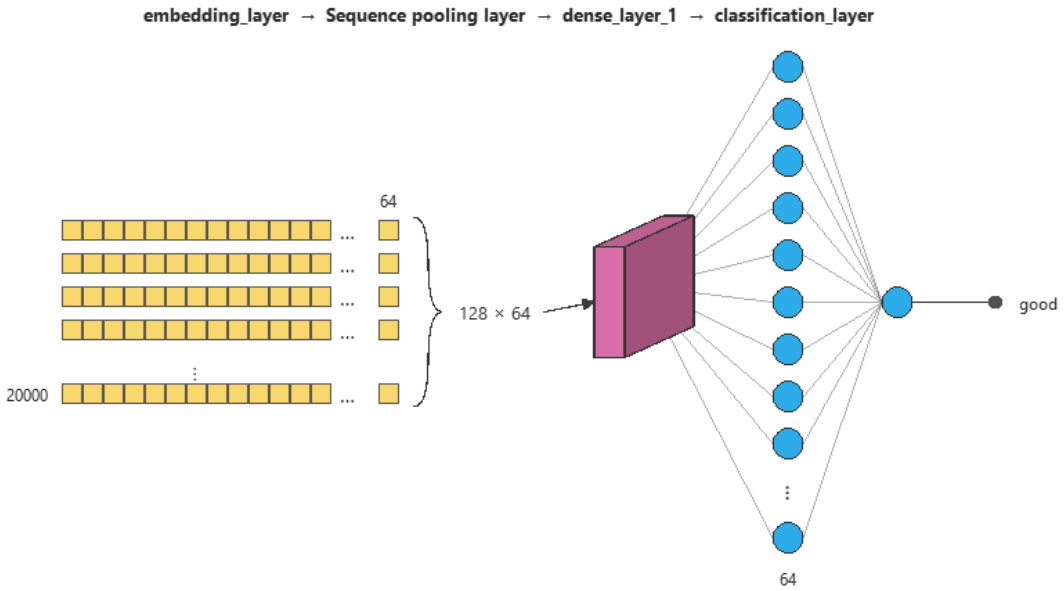


Figura 2. La red del modelo C en Neural Designer: embedding de 20.000 palabras en 64 dimensiones, pooling, capa densa de 64 neuronas y salida de clasificación.

4.3 Estrategia de entrenamiento

- **Error:** entropía cruzada, con regularización L2 (0,001).
- **Optimizador:** Adam, tasa de aprendizaje 0,001, lotes de 64 reseñas.
- **Parada temprana:** el entrenamiento se detiene tras 5 épocas sin mejorar el error de validación, y se conservan los parámetros de la mejor época.

Estas opciones salen del primer entrenamiento, sin regularización ni parada temprana: el error de validación tocó su mínimo en la época 6 y luego subió mientras el de entrenamiento seguía bajando, un sobreajuste claro. Las otras 69 épocas no sirvieron de nada.

Con la configuración final, el modelo C se detuvo en la época 22 y conservó la 17 (figura 3). El error de validación se estabiliza en torno a 0,28 a partir de la quinta época, mientras el de entrenamiento sigue bajando despacio hasta 0,24: la distancia entre las dos curvas es pequeña, sin el sobreajuste del primer entrenamiento. Todo el entrenamiento duró unos segundos en una GPU RTX 5070 Ti.

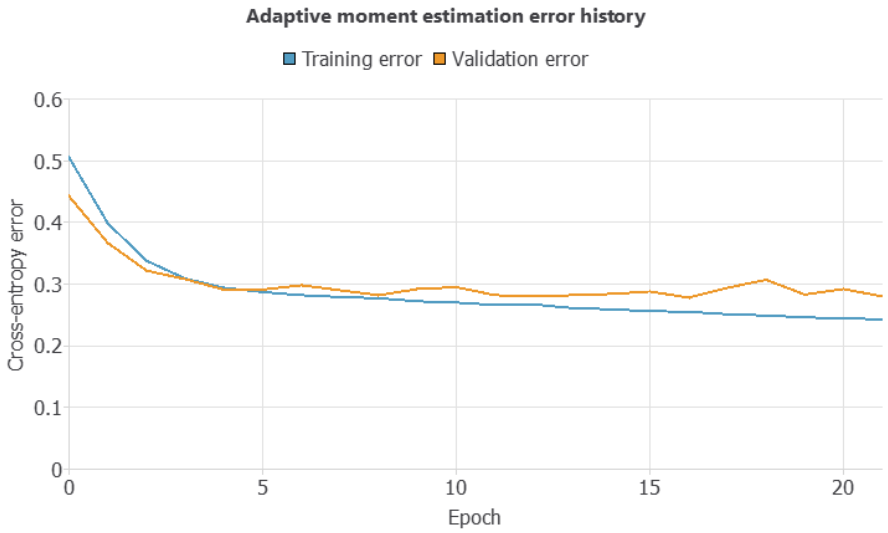


Figura 3. Historial del error (entropía cruzada) de entrenamiento y de validación por época, con el optimizador Adam, en Neural Designer. La parada temprana corta cuando el error de validación deja de mejorar.

4.4 Comparación de modelos

Se compararon tres redes con la misma configuración (tabla 3) y un modelo de referencia clásico: TF-IDF con pares de palabras y regresión logística (scikit-learn), entrenado con el mismo archivo y las mismas proporciones.

Tabla 3. Modelos entrenados en Neural Designer. Exactitud y recall (porcentaje de cada clase que el modelo reconoce) en el subconjunto de prueba.

	Datos	Red	Épocas (mejor)	Exactitud	Recall mala / buena	AUC	Referencia TF-IDF
A	10.000 por clase	con atención	16 (11)	84,8 %	86,6 % / 83,1 %	0,922	88,2 %
B	40.000 por clase	con atención	14 (9)	88,6 %	88,3 % / 88,8 %	0,956	90,0 %
C	40.000 por clase	**sin atención**	22 (17)	**88,4 %**	88,3 % / 88,6 %	0,953	90,0 %

- **Lo que más importa son los datos.** De 10.000 a 40.000 reseñas por clase la exactitud sube casi 4 puntos, y la distancia con la referencia baja de 3,4 a 1,4 puntos.
- **La atención no aporta nada aquí.** B y C se diferencian en 20 reseñas de 16.000, dentro del margen de error. Las reseñas cortas se clasifican por las palabras que contienen más que por su orden.
- **La referencia clásica sigue 1,4 puntos por delante.** Con pares de palabras captura directamente expresiones como *no recomiendo*. Es un resultado habitual: una red entrenada desde cero necesita más datos que un modelo de bolsa de palabras.

Se eligió el **modelo C**: la misma exactitud que B con cuatro capas en lugar de seis, más rápido en la CPU del despliegue (el coste de la atención crece con el cuadrado de la longitud del texto). Y un mensaje claro: el modelo sencillo es tan bueno como el de atención.

4.5 Resultados del modelo C

En las 16.000 reseñas de prueba, el modelo C acierta el 88,4 % y reconoce por igual las dos clases (figura 4): el 88,6 % de las buenas y el 88,3 % de las malas. Se equivoca en 920 reseñas buenas y 928 malas, un error equilibrado. El área bajo la curva ROC es 0,953.

	Predicted positive	Predicted negative	Total
Actual positive	**7172 (88.6%)**	920 (11.4%)	8092 (100.0%)
Actual negative	928 (11.7%)	**6980 (88.3%)**	7908 (100.0%)
Total	8100 (50.6%)	7900 (49.4%)	16000 (100.0%)

Figura 4. Matriz de confusión del modelo C en Neural Designer, en las 16.000 reseñas de prueba. La clase positiva es *buena* y la negativa, *mala*.

5 Inferencia con OpenNN

El paquete de despliegue de Neural Designer incluye el motor completo de la aplicación, demasiado pesado para un repositorio o un contenedor. Como Neural Designer está construido sobre OpenNN, la red entrenada se puede ejecutar con OpenNN directamente.

Exportación. Un proyecto de Neural Designer (`.nd`) es un contenedor HDF5 con la red en el formato de OpenNN, sus parámetros en binario y el vocabulario en la configuración. El script `export_model.py` extrae cuatro archivos (5,3 MB): la red, los pesos, el vocabulario y el umbral de decisión.

Predictor. `predict.cpp` es un programa de unas 100 líneas enlazado con OpenNN y compilado solo para CPU: el ejecutable ocupa 1,3 MB. CMake descarga OpenNN de GitHub en la versión exacta con la que se compiló Neural Designer. Un envoltorio en Python normaliza el texto, llama al predictor y devuelve las probabilidades. Las reseñas sin palabras (solo estrellas o emojis) no reciben predicción en lugar de una conjetura.

Validación. En las 16.000 reseñas de prueba, el área bajo la curva ROC es 0,9532 en OpenNN y 0,953 en Neural Designer, y las matrices de confusión difieren como mucho en 2 reseñas por celda, por la aritmética de GPU frente a CPU. En 3.993 reseñas que ninguna de las dos herramientas había visto, la exactitud es del 88,5 % (AUC 0,955), y las clasifica en 0,2 segundos en CPU.

Umbral de decisión. Con el umbral 0,5 el modelo reconoce mejor las reseñas malas que las buenas. Con 0,44 las dos clases quedan equilibradas, con la misma exactitud, y se reproduce el umbral de Neural Designer (tabla 4). El umbral se guarda con el modelo, no en el código.

Tabla 4. Efecto del umbral en las 3.993 reseñas no vistas.

Umbral	Exactitud	Recall mala	Recall buena
0,50	88,5 %	90,6 %	86,4 %
0,44	88,5 %	88,2 %	88,8 %

6 Recogida de reseñas

6.1 Explorar Google Maps

La primera idea era un scraper propio con Playwright, un navegador automatizado. La exploración, sin cuenta de Google, mostró que Maps ofrece una *vista limitada*: solo 5 reseñas. Para ver más, u ordenarlas, pide iniciar sesión. También mostró que los datos internos de la página llevan la fecha exacta de cada reseña, aunque la página solo diga *hace 9 meses*.

Tabla 5. Opciones para obtener las reseñas.

Opción	Reseñas	Inconveniente
Scraper propio sin cuenta	5 por negocio	Inútil para hacer estadísticas
Scraper con una cuenta de Google	Todas	Automatizar una cuenta es lo primero que Google vigila (bloqueos, CAPTCHAs)
APIs oficiales (Places, TripAdvisor, Yelp)	3–5, o de pago	Insuficiente
API de Google Business Profile	Todas	Solo para el propietario del negocio
Apify, plan gratuito	Todas	5 $ de crédito al mes

Se revisaron otras webs de opiniones: Google concentra casi todas las reseñas de comercios (TripAdvisor, Foursquare o Páginas Amarillas apenas tienen), y Trustpilot prohíbe expresamente el acceso automatizado. Una regla se mantuvo en todo momento: nunca evitar CAPTCHAs ni bloqueos, ni disfrazar el navegador.

6.2 Apify sin gastar nada

Apify es una plataforma de extracción de datos con un plan gratuito de 5 $ de crédito al mes. Se usó con una regla de coste cero: la cuenta no tiene ningún medio de pago, así que no se le puede cobrar nada; cuando se agota el crédito, las descargas se detienen hasta el mes siguiente.

- El actor oficial de reseñas de Google Maps, con la opción que descarta en origen el nombre, el perfil y la foto de cada autor. Nunca llega ningún dato personal.
- Una prueba con 50 reseñas fijó el precio real: 0,0006 $ por reseña.
- La descarga completa trajo **7.490 reseñas por 4,49 $**, de octubre de 2021 a octubre de 2026. El límite de reseñas llegó antes de 2019, que era la fecha objetivo.
- El código admite límites de reseñas y de gasto por descarga como parámetros: quien quiera pagar puede subirlos.

Un incidente útil. Durante la descarga larga, un error de red cortó el script local, pero la descarga siguió en los servidores de Apify. Lanzarla de nuevo habría pagado dos veces. Se localizó por la API y se recuperó sin coste. Desde entonces, el cliente reintenta las lecturas tras un error de red, nunca reintenta el inicio de una descarga y puede retomar una descarga existente por su identificador.

7 Base de datos y pipeline

Las descargas se guardan como JSON y se cargan en una base de datos con SQLAlchemy, que funciona igual con **SQLite** (un archivo local, sin instalar nada) y con **PostgreSQL** (en Docker).

- `places`: los negocios. `runs`: cada descarga, con su coste.
- `reviews`: fecha exacta, estrellas, texto e idioma. Sin datos de los autores.

- `predictions`: una fila por reseña y modelo, para comparar modelos futuros sin perder los resultados.
- La vista `review_facts`: una fila por reseña con todo lo que necesita Power BI.

El pipeline tiene tres pasos: descargar, importar y clasificar. Importar dos veces no duplica nada, y clasificar solo procesa las reseñas en español que el modelo aún no ha visto.

8 Resultados en El Corte Inglés Princesa

8.1 Los datos

Las 7.490 reseñas tienen una media de 4,20 estrellas. Solo el 45 % lleva texto: 3.071 en español, 302 en otros idiomas, y el resto solo tiene estrellas. El modelo clasificó 3.063 textos en español (los otros 8 solo tenían emojis o símbolos). Son textos cortos: la mediana es de 11 palabras, la mitad que en las reseñas de Amazon.

8.2 ¿Funciona el modelo fuera de Amazon?

El modelo se entrenó con reseñas de productos y ahora lee reseñas de unos grandes almacenes. Las estrellas de Google dan una etiqueta gratis: las de 1–2 estrellas deberían ser malas y las de 4–5, buenas. En las 2.796 reseñas clasificadas con esas estrellas, el modelo coincide con ellas en el 87,1 %: solo pierde 1,4 puntos respecto a las reseñas de Amazon que no había visto (tabla 6).

Tabla 6. El modelo C en su dominio de entrenamiento y en reseñas reales del negocio.

	Amazon, no vistas (3.993)	El Corte Inglés (2.796)
Exactitud / coincidencia con las estrellas	88,5 %	87,1 %
Recall mala / buena	88,2 % / 88,8 %	86,6 % / 87,2 %
AUC	0,955	0,940

Un estudio previo con el modelo de referencia ya lo anticipaba: entrenado con Amazon, acierta el 90 %–92 % en reseñas reales de hoteles. Lo difícil no es el sector sino el estilo: con reseñas sarcásticas el acierto cae al 47 %, como lanzar una moneda.

8.3 Las opiniones negativas crecen

Tabla 7. Evolución anual de las reseñas.

Año	Reseñas	Nota media	Con 1–2 estrellas	Malas según el modelo
2021 (desde el 29/10)	514	4,2	6,6 %	22,4 %
2022	2.371	4,3	7,2 %	24,1 %
2023	1.927	4,2	8,8 %	30,1 %
2024	1.180	4,1	11,8 %	32,0 %
2025	927	4,2	13,1 %	32,5 %
2026 (hasta el 24/10)	571	4,1	12,6 %	34,4 %

La proporción de reseñas de 1–2 estrellas casi se duplica entre 2021–2022 y 2025–2026, y el modelo ve la misma tendencia en los textos: del 22 % al 34 % de reseñas malas (tabla 7). La nota media apenas se mueve, de 4,3 a 4,1: justo lo que la media no cuenta.

8.4 Quién escribe se queja más

Las cifras del modelo y las de las estrellas no son comparables directamente, porque miden reseñas distintas. Quien escribe un texto se queja mucho más que quien solo pone estrellas (tabla 8). La aplicación lo explica en su propia sección.

Tabla 8. Estrellas según el tipo de reseña.

Tipo de reseña	Reseñas	Nota media	Con 1–2 estrellas
Texto en español	3.071	3,95	17,3 %
Texto en otro idioma	302	4,28	9,6 %
Solo estrellas	4.117	4,38	3,6 %

8.5 De qué hablan

Para encontrar las palabras que distinguen las reseñas malas de las buenas se usó la razón de log-odds con un prior de Dirichlet informativo [2]. A diferencia de la frecuencia simple, ignora las palabras comunes a todas las reseñas y no deja que una palabra vista dos veces encabece la lista.

- **Reseñas malas:** *cliente* (atención al cliente), *pedido*, *online*, *nadie*, *minutos*, *días*, *recoger*. Las quejas giran en torno a la atención y a los pedidos en línea y su recogida.
- **Reseñas buenas:** *variedad*, *calidad*, *productos*, *trato*, *amable*.

9 Aplicación web

La aplicación, hecha con **Streamlit** y gráficos de **Altair**, está en español y funciona con varios negocios. La página de inicio muestra una tarjeta por negocio (figura 5) y un botón para crear el dashboard de uno nuevo.



Figura 5. Página de inicio: una tarjeta por negocio con su nota, las reseñas analizadas y el porcentaje de reseñas malas en los últimos 12 meses.

Crear un dashboard nuevo. El diálogo (figura 6) busca el negocio en Google Maps, sin coste y sin cuenta, deja elegir cuántas reseñas descargar y muestra el coste estimado y el crédito de Apify que queda en el mes. Al confirmar, descarga las reseñas, las clasifica y abre el dashboard. Cada dashboard tiene un botón para actualizar a mano, que descarga solo las reseñas nuevas.

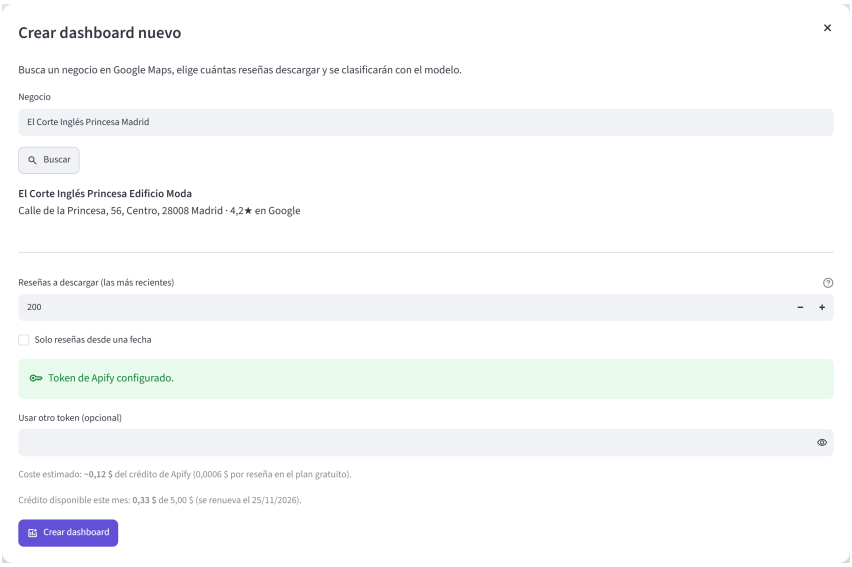


Figura 6. Diálogo para crear un dashboard: búsqueda del negocio, número de reseñas y coste estimado.

El dashboard de un negocio. Responde a cuatro preguntas: cómo han ido los últimos 12 meses respecto a los anteriores (figura 7), cómo evolucionan las opiniones (figura 8), si el modelo coincide con las estrellas (figura 9) y de qué hablan las reseñas (figura 10). Termina con ejemplos recientes en fragmentos cortos.

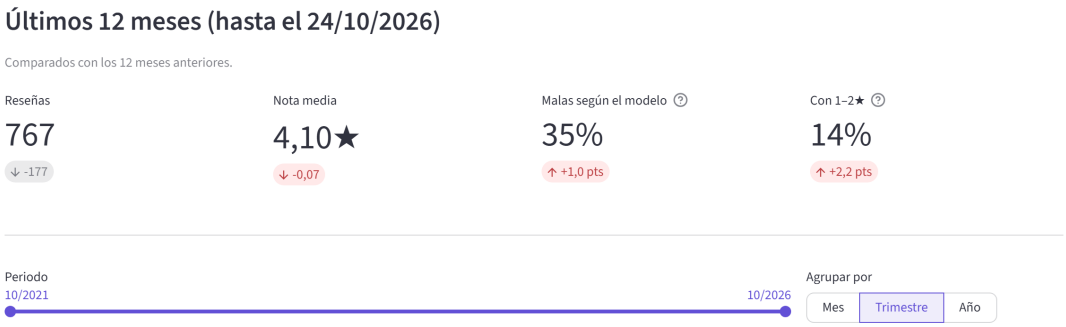


Figura 7. Últimos 12 meses frente a los 12 anteriores, y los filtros de periodo y agrupación.

Diseño de los gráficos. Los colores salen de una paleta validada para daltonismo, con tema claro y oscuro. Cada color tiene un significado fijo: azul para el modelo y naranja para las estrellas, azul y rojo para bueno y malo, gris para el contexto. Ningún gráfico tiene dos ejes verticales: el volumen va en un gráfico aparte. El pie de la aplicación enlaza el código fuente, como pide la licencia AGPL.

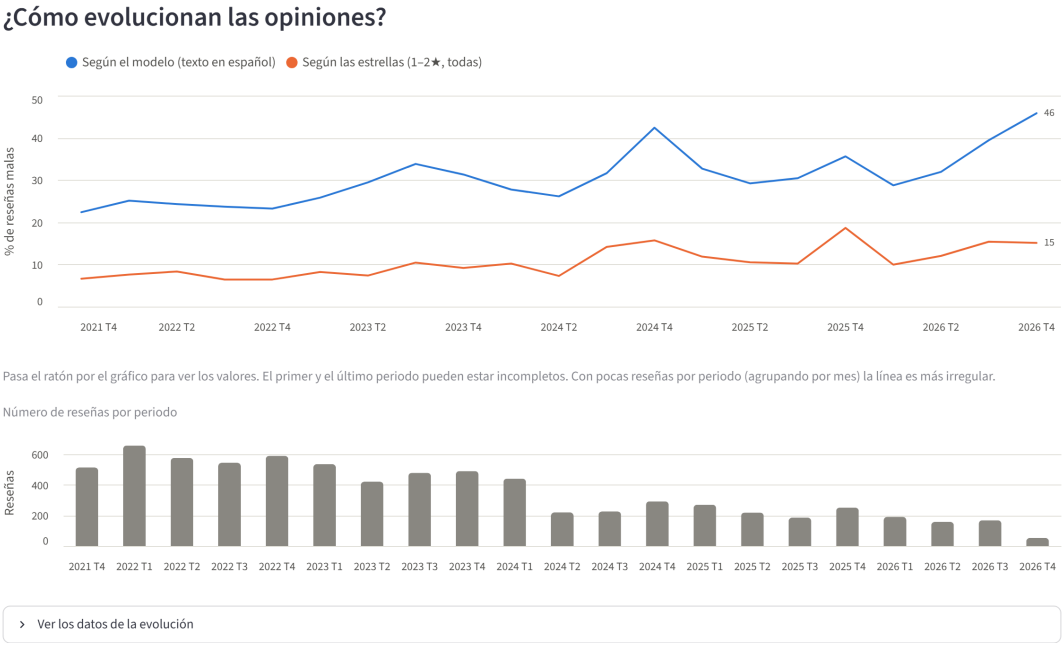


Figura 8. Evolución por trimestre: reseñas malas según el modelo (azul) y con 1–2 estrellas (naranja) en un solo eje, y el volumen de reseñas en un gráfico aparte.

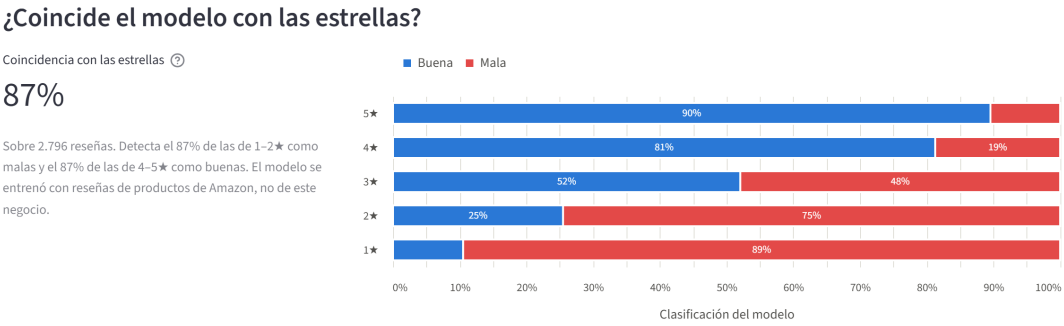


Figura 9. Coincidencia del modelo con las estrellas: cómo clasifica el modelo las reseñas de cada número de estrellas.

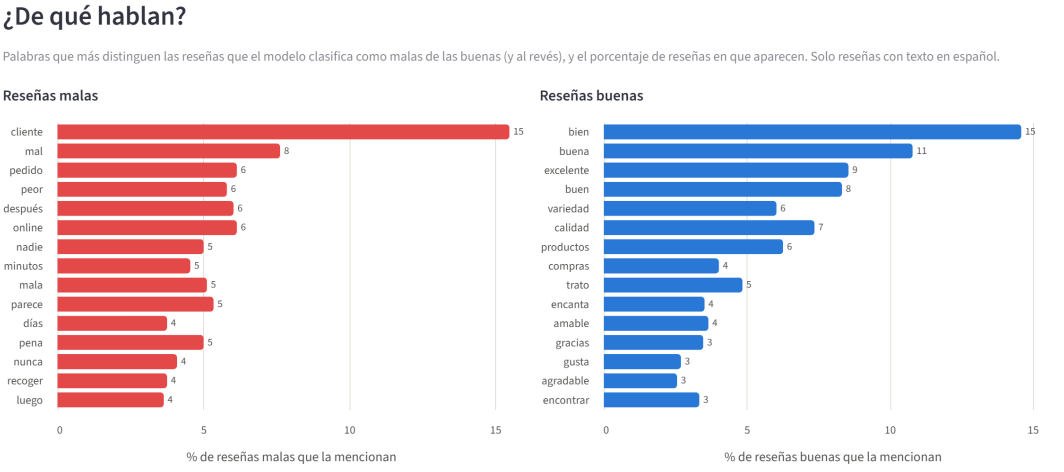


Figura 10. Palabras que más distinguen las reseñas malas de las buenas, y al revés.

10 Informe de Power BI

Power BI Desktop, gratuito, se conecta a la base de datos PostgreSQL de Docker en modo **DirectQuery**: no copia los datos, sino que consulta la base de datos cada vez que se actualiza, lo que permite la actualización automática de la página cada pocos minutos. Lee la vista `review_facts` y calcula las cifras con cuatro medidas DAX, equivalentes a las de la aplicación:

```
Reseñas           = COUNTROWS ( review_facts )
Nota media        = AVERAGE ( review_facts[stars] )
% malas (modelo) = DIVIDE ( SUM ( review_facts[model_bad] ), COUNT ( review_facts[model_bad] ) )
% 1-2 estrellas  = DIVIDE ( SUM ( review_facts[low_stars] ), COUNTROWS ( review_facts ) )
```

COUNT ignora los valores vacíos, así que el porcentaje del modelo solo cuenta las reseñas clasificadas, igual que la aplicación.

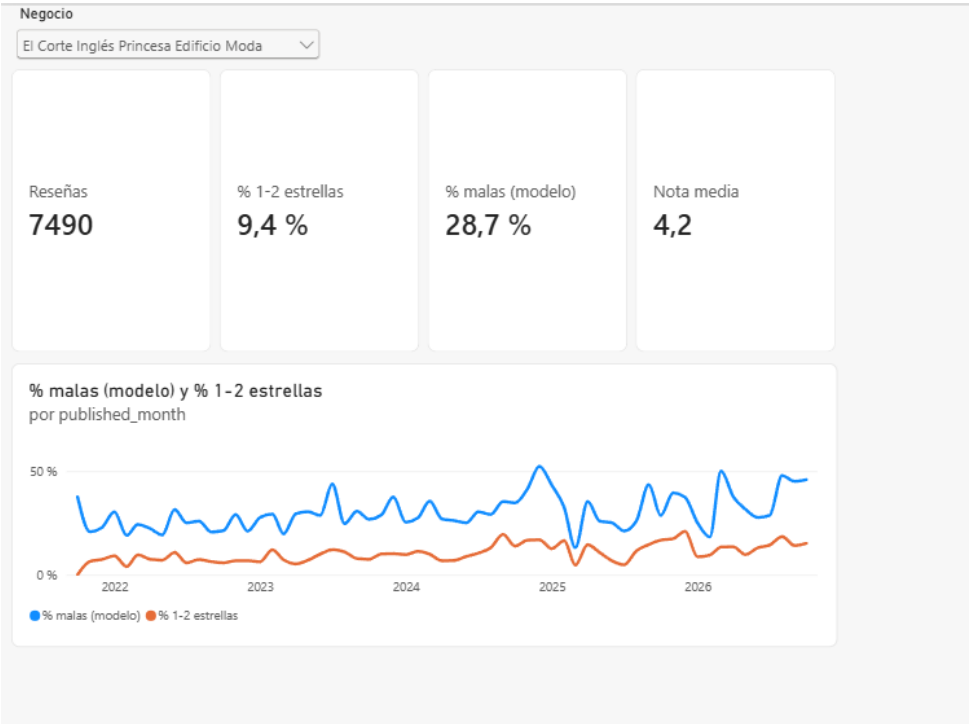


Figura 11. Informe de Power BI sobre PostgreSQL: selector de negocio, cifras clave y evolución mensual.

11 Despliegue con Docker

Todo el sistema arranca con un comando:

```
docker compose up --build
```

Docker Compose levanta dos contenedores (figura 12):

- **db**: PostgreSQL 17, con los datos en un volumen persistente.
- **app**: la aplicación Streamlit. Se construye en dos etapas. La primera compila el predictor de OpenNN desde el código fuente (Debian con GCC 14 y CMake). La segunda solo lleva Python 3.13, las dependencias fijadas con uv, Chromium para la búsqueda de negocios, el modelo y el ejecutable del predictor.

Al arrancar, la aplicación crea las tablas, importa las descargas JSON que aún no están en la base de datos, las clasifica y abre la web. Una base de datos nueva se llena sola. El token de Apify y la contraseña se leen de un archivo `.env` que no se sube al repositorio.

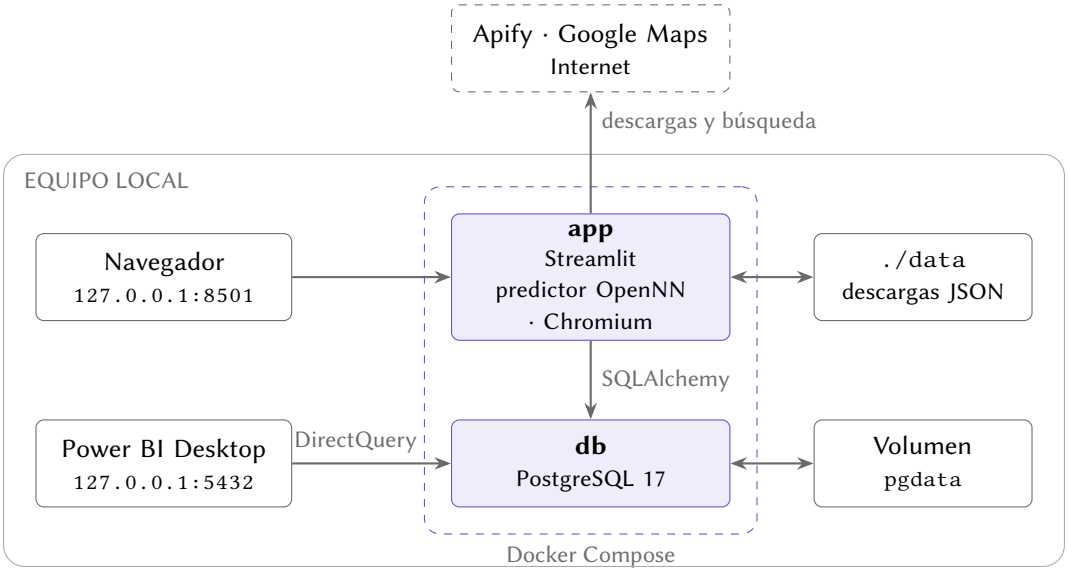


Figura 12. Despliegue con Docker Compose. Los puertos solo escuchan en el propio equipo (`127.0.0.1`).

Lo que enseñaron las pruebas.

- La primera construcción falló: compilar OpenNN con paralelismo ilimitado lanzó 28 compilaciones a la vez, cada una puede pasar de 1 GB, y el motor de Docker se quedó sin memoria. Ahora usa 4 trabajos por defecto, ampliables con un argumento.
- Docker publica los puertos en todas las interfaces de red por defecto, lo que habría expuesto PostgreSQL, con su contraseña por defecto, a la red. Los dos puertos escuchan ahora solo en `127.0.0.1`.
- El predictor compilado con GCC en Linux da exactamente las mismas predicciones que el compilado con MSVC en Windows: 2.185 reseñas buenas y 878 malas en ambos.

12 Conclusiones y trabajo futuro

12.1 Lo aprendido

- **Los datos pesan más que la arquitectura.** Cuadruplicar los datos subió casi 4 puntos; añadir atención, nada.
- **El modelo sencillo gana.** Sin atención, la red es igual de precisa, más pequeña y más rápida en CPU.
- **El preprocesado tiene que ser idéntico** al entrenar y al predecir. El problema de los acentos solo apareció al inspeccionar el vocabulario, y la paridad entre Neural Designer y OpenNN se comprobó reseña a reseña.
- **Las estrellas son etiquetas gratis** para medir el modelo en su dominio real: el 87,1 % de coincidencia confirma que un modelo de productos sirve para unos grandes almacenes.
- **Las restricciones dan forma al diseño.** Coste cero y privacidad descartaron el scraping con cuenta y llevaron a Apify sin medio de pago, con límites de gasto y descargas recuperables.

- **Documentar cada decisión** con sus alternativas ha hecho posible escribir esta memoria.

12.2 Trabajo futuro

- Clasificar textos sin estrellas (foros, redes sociales), el objetivo de fondo del modelo de texto.
- Un tokenizador de OpenNN que entienda UTF-8, para no tener que normalizar el texto.
- Entrenar con reseñas del propio dominio (comercios, restaurantes) para recuperar el punto y medio perdido y mejorar con reseñas indirectas o sarcásticas.
- Analizar los picos de reseñas malas: rebajas, Black Friday, Navidad, obras en el edificio o el nuevo servicio de atención con inteligencia artificial, que aparece en los ejemplos recientes.

Licencia, datos y reproducibilidad

El código se publica bajo la licencia **GNU AGPL v3.0**, la misma que OpenNN, que el predictor enlaza. Los datos de entrenamiento no se incluyen en el repositorio: cada conjunto conserva su licencia, y el repositorio explica cómo reconstruirlos. El modelo publicado deriva del corpus de Amazon y solo puede usarse con fines no comerciales. Los proyectos de Neural Designer no se publican, porque sus informes contienen textos de reseñas.

Para ejecutar el proyecto basta con Docker Desktop y un token gratuito de Apify para descargar reseñas:

```
git clone https://github.com/miguelbdiez/impacto
cd impacto
cp .env.example .env        # token de Apify (opcional)
docker compose up --build   # http://localhost:8501
```

Referencias

[1] P. Keung, Y. Lu, G. Szarvas y N. A. Smith. *The Multilingual Amazon Reviews Corpus*. Proceedings of EMNLP 2020.

[2] B. L. Monroe, M. P. Colaresi y K. M. Quinn. *Fightin' Words: Lexical Feature Selection and Evaluation for Identifying the Content of Political Conflict*. Political Analysis, 16(4), 2008.

[3] Neural Designer. https://www.neuraldesigner.com

[4] OpenNN. https://github.com/Artelnics/opennn